



UNIVERSIDAD DE COSTA RICA

Sede del Atlántico

RP Recinto de Paraíso

UNIVERSIDAD DE COSTA RICA
SEDE DEL ATLÁNTICO - RECINTO PARAÍSO
CARRERA DE INFORMÁTICA EMPRESARIAL
CURSO: IF0009 - Desarrollo de Software IV
PROFESOR: Mag. Jonathan Granados C.
SEMESTRE: II-2026

Taller 2: Control de Versiones con Git/GitHub y Maquetación Web Responsiva (Portafolio Académico)

Metadatos

- **Tiempo Estimado:** 3 horas
- **Herramientas Requeridas:**
 - Git instalado en su sistema operativo.
 - Cuenta de usuario activa en [GitHub](#).
 - Visual Studio Code u otro editor de código de su elección.
 - Navegador web moderno (Google Chrome, Mozilla Firefox, Edge, etc.).
- **Metas de Aprendizaje:**
 - Configurar un repositorio Git local, registrando el historial del proyecto a través de confirmaciones (commits) semánticas y controlando qué archivos ignorar.
 - Desarrollar una página web responsiva ("Portafolio Académico") usando etiquetas semánticas de HTML5 y una grilla dinámica en CSS Grid/Flexbox.
 - Sincronizar el repositorio local con un servidor remoto en GitHub y desplegar de manera pública la aplicación web mediante el servicio gratuito GitHub Pages.

Introducción (Lectura y Conceptos)

En el ciclo de vida del desarrollo web moderno, no basta con saber escribir código local; es indispensable dominar las herramientas de control de versiones y los mecanismos de despliegue continuo en la nube.

- **Control de Versiones (Git):** Es un sistema que rastrea los cambios realizados en el código a lo largo del tiempo. Le permite guardar instantáneas (commits) del estado de su proyecto, revertir a versiones anteriores si algo falla y trabajar de forma segura en ramas alternativas (branches) sin alterar el código de producción.
- **Plataformas de Alojamiento (GitHub):** Actúa como un servidor remoto en la nube para hospedar sus repositorios Git. Facilita la colaboración mediante Pull Requests, permite auditar el código y ofrece servicios integrados de hosting como **GitHub Pages** para publicar sitios web estáticos (HTML, CSS y JavaScript) directamente desde una rama del repositorio.

- **Diseño Responsivo con CSS Grid:** Es un sistema de diseño bidimensional diseñado para la maquetación web. A diferencia de Flexbox (que es principalmente unidireccional), Grid le permite distribuir elementos en filas y columnas simultáneamente, adaptándose de forma automática a pantallas de móviles, tabletas y computadoras mediante la directiva `auto-fit` y las unidades de fracción (`fr`).

En este taller guiado, usted creará un portafolio web personal, registrará sus avances con Git, lo publicará en GitHub y lo desplegará en la nube para que cualquiera pueda acceder a él mediante una URL pública.

Parte 1: Práctica Guiada (Paso a Paso)

Paso 1: Estructura Semántica del Portafolio (HTML5)

Cree una nueva carpeta en su computadora llamada `portafolio-academico`. Dentro de ella, cree un archivo llamado `index.html`.

Escriba la siguiente estructura semántica básica que define las secciones principales del portafolio (Cabecera, Presentación, Grilla de Proyectos y Pie de Página):

[!NOTE] La etiqueta `<article>` se utiliza para agrupar contenido autónomo e independiente (como una tarjeta de producto, una entrada de blog o un proyecto del portafolio), facilitando la lectura del sitio a personas que utilicen lectores de pantalla.

```
<!DOCTYPE html>
<html lang="es">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Portafolio Académico - UCR</title>
    <link rel="stylesheet" href="styles.css">
  </head>
  <body>
    <!-- Encabezado de Navegación -->
    <header class="navbar">
      <div class="logo">Portafolio Académico</div>
```

```

        <nav class="nav-links">
            <a href="#sobre-mi">Sobre Mí</a>
            <a href="#proyectos">Proyectos</a>
            <a href="https://github.com"
                target="_blank"
                class="btn-github">Mi GitHub</a>
        </nav>
    </header>

    <!-- Sección de Héroe (Sobre Mí) -->
    <section id="sobre-mi" class="hero-section">
        <div class="hero-content">
            <h1>Hola, soy <span class="highlight">Estudiante UCR</span></h1>
            <p class="subtitle">
                Estudiante de Informática Empresarial |
                Futuro Full-Stack Developer
            </p>
            <p class="description">
                Bienvenido a mi espacio de proyectos académicos.
                Aquí registro el desarrollo de las aplicaciones
                e investigaciones de la carrera en el Recinto de Paraíso.
            </p>
        </div>
    </section>

    <!-- Sección de Proyectos (CSS Grid) -->
    <main id="proyectos" class="projects-section">
        <h2>Mis Proyectos Académicos</h2>
        <div class="projects-grid">

            <!-- Tarjeta de Proyecto 1 -->
            <article class="project-card">
                <div class="project-badge">Back-end</div>
                <h3>Taller 1: Servidor HTTP Java</h3>
                <p>
                    Implementación desde cero de un servidor HTTP local
                    nativo utilizando las librerías del JDK para
                    entender el flujo cliente-servidor.
                </p>
                <div class="project-tech">
                    <span>Java</span>
                    <span>HTML</span>
                    <span>CSS</span>
                </div>
            </article>

```

```

        <!-- Tarjeta de Proyecto 2 -->
        <article class="project-card">
            <div class="project-badge font-design">Diseño</div>
            <h3>Laboratorio 1: Configuración & Git</h3>
            <p>
                Establecimiento del entorno de desarrollo global e
                introducción a los comandos esenciales de consola
                para el control de versiones.
            </p>
            <div class="project-tech">
                <span>Git</span>
                <span>Bash</span>
                <span>Node.js</span>
            </div>
        </article>
    ]

    <!-- Tarjeta de Proyecto 3 -->
    <article class="project-card">
        <div class="project-badge font-api">API</div>
        <h3>Proyecto Integrador (Avance 1)</h3>
        <p>
            Planificación de la base de datos relacional y
            modelado inicial de una API RESTful para el
            sistema empresarial acumulativo.
        </p>
        <div class="project-tech">
            <span>SQL</span>
            <span>REST</span>
            <span>JSON</span>
        </div>
    </article>

</div>
</main>

<!-- Pie de Página -->
<footer class="app-footer">
    <p>© 2026 Universidad de Costa Rica - Informática Empresarial</p>
    <p>Desarrollo de Software IV | Recinto de Paraíso</p>
</footer>

</body>
</html>

```

Paso 2: Estilización y Responsividad (CSS3 con Grid y Flexbox)

En la misma carpeta, cree un archivo llamado `styles.css`. Utilizaremos variables CSS para establecer una paleta de colores oscuros de alta calidad (estilo Dark Mode) y CSS Grid con la propiedad `repeat(auto-fit, ...)` para que la distribución de tarjetas se ajuste solapadamente al ancho disponible en pantalla.

```
/* Variables CSS y Configuración Base */
:root {
  --bg-primary: #0f172a;
  --bg-secondary: #1e293b;
  --text-main: #f8fafc;
  --text-muted: #94a3b8;
  --color-accent: #38bdf8;
  --color-accent-hover: #0ea5e9;
  --border-color: #334155;
  --badge-bg: #0369a1;
  --font-family: -apple-system, BlinkMacSystemFont,
    "Segoe UI", Roboto, sans-serif;
}

* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

body {
  background-color: var(--bg-primary);
  color: var(--text-main);
  font-family: var(--font-family);
  line-height: 1.6;
}

/* Barra de Navegación con Flexbox */
.navbar {
  background-color: var(--bg-secondary);
  border-bottom: 1px solid var(--border-color);
  padding: 15px 30px;
  display: flex;
  justify-content: space-between;
  align-items: center;
  position: sticky;
  top: 0;
  z-index: 100;
}
```

```

}

.navbar .logo {
  font-weight: bold;
  font-size: 1.25rem;
  color: var(--color-accent);
}

.nav-links a {
  color: var(--text-main);
  text-decoration: none;
  margin-left: 20px;
  font-size: 0.95rem;
  transition: color 0.2s ease;
}

.nav-links a:hover {
  color: var(--color-accent);
}

.nav-links .btn-github {
  background-color: var(--color-accent);
  color: var(--bg-primary) !important;
  font-weight: 600;
  padding: 6px 12px;
  border-radius: 4px;
  transition: background-color 0.2s ease;
}

.nav-links .btn-github:hover {
  background-color: var(--color-accent-hover);
}

/* Sección de Héroe */
.hero-section {
  padding: 80px 40px;
  max-width: 1000px;
  margin: 0 auto;
  border-bottom: 1px solid var(--border-color);
}

.hero-content h1 {
  font-size: 3rem;
  margin-bottom: 10px;
}

```

```

.hero-content .highlight {
  color: var(--color-accent);
}

.hero-content .subtitle {
  font-size: 1.25rem;
  color: var(--text-muted);
  margin-bottom: 20px;
}

.hero-content .description {
  font-size: 1.1rem;
  color: var(--text-muted);
  max-width: 700px;
}

/* Sección de Proyectos - Distribución con CSS Grid */
.projects-section {
  padding: 60px 40px;
  max-width: 1200px;
  margin: 0 auto;
}

.projects-section h2 {
  font-size: 2rem;
  margin-bottom: 30px;
  text-align: center;
  color: var(--text-main);
}

/* Grilla Responsiva sin necesidad de Media Queries rígidos */
.projects-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(280px, 1fr));
  gap: 25px;
}

/* Tarjeta de Proyecto */
.project-card {
  background-color: var(--bg-secondary);
  border: 1px solid var(--border-color);
  border-radius: 8px;
  padding: 24px;
  display: flex;
  flex-direction: column;
  position: relative;

```



```

    transition: transform 0.2s ease, border-color 0.2s ease;
}

.project-card:hover {
    transform: translateY(-5px);
    border-color: var(--color-accent);
}

.project-badge {
    background-color: var(--badge-bg);
    color: var(--text-main);
    font-size: 0.75rem;
    font-weight: bold;
    padding: 4px 8px;
    border-radius: 4px;
    align-self: flex-start;
    margin-bottom: 15px;
    text-transform: uppercase;
}

.project-badge.font-design { background-color: #047857; }
.project-badge.font-api { background-color: #7c3aed; }

.project-card h3 {
    font-size: 1.25rem;
    margin-bottom: 10px;
    color: var(--text-main);
}

.project-card p {
    color: var(--text-muted);
    font-size: 0.95rem;
    margin-bottom: 20px;
    flex-grow: 1; /* Ocupa el espacio vertical disponible */
}

.project-tech {
    display: flex;
    gap: 8px;
}

.project-tech span {
    background-color: var(--bg-primary);
    border: 1px solid var(--border-color);
    font-size: 0.8rem;
    padding: 3px 8px;
}

```

```

border-radius: 4px;
color: var(--color-accent);
}

/* Pie de Página */
.app-footer {
background-color: var(--bg-secondary);
border-top: 1px solid var(--border-color);
padding: 25px 30px;
text-align: center;
font-size: 0.85rem;
color: var(--text-muted);
}

.app-footer p {
margin: 5px 0;
}

```

Verifique el diseño abriendo directamente el archivo `index.html` en su navegador web.

Paso 3: Inicialización e Historial de Cambios (Git)

Abra la consola de comandos de su sistema (Terminal en macOS/Linux, Git Bash o PowerShell en Windows) e ingrese a la ruta donde tiene guardada la carpeta del proyecto.

[!IMPORTANT] Antes de inicializar el repositorio, configure su identidad global si no lo ha hecho anteriormente en su equipo personal. Esto asegura que cada confirmación registre su autoría de forma correcta.

1. **Configure su usuario y correo electrónico:** `bash git config --global user.name "Su Nombre Completo" git config --global user.email "su-correo-institucional@ucr.ac.cr"`
2. **Inicialice el repositorio local:** `bash git init` (Observará que se crea una carpeta oculta `.git` en el directorio).
3. **Cree un archivo `.gitignore`:** En desarrollo profesional, hay archivos temporales del sistema u carpetas de configuración del editor que no deben subirse al repositorio público. Cree un archivo llamado `.gitignore` en la raíz y agregue el siguiente contenido: text
`.vscode/ Thumbs.db .DS_Store *.log`

4. **Revise el estado del repositorio:** `bash git status` *Debería ver enlistados los archivos `index.html`, `styles.css` y `.gitignore` en la sección de archivos no rastreados (Untracked).*
5. **Agregue los archivos al área de preparación (Stage):** `bash git add .`
6. **Realice la primera confirmación (Commit):** `bash git commit -m "feat: portafolio academico responsivo"`

Paso 3.1: Uso de Git en Visual Studio Code y Google Antigravity

Aunque la terminal es el estándar de oro para el control de versiones, Visual Studio Code y el asistente de Inteligencia Artificial Google Antigravity ofrecen herramientas visuales e inteligentes que facilitan el flujo de trabajo.

1. Gestión Visual desde Visual Studio Code (UI de Source Control)

Para trabajar con Git de manera visual dentro del editor, siga estos pasos:

- **Acceda al Panel de Control de Fuentes:** En la barra lateral izquierda, haga clic en el icono de **Source Control** (tres nodos conectados por líneas, o presione `Ctrl + Shift + G`).
- **Agregar al Área de Preparación (Stage):** En la sección de *Changes* (Cambios), coloque el mouse sobre un archivo (como `index.html`) y haga clic en el botón **+** (Stage Changes) para agregarlo al Stage (equivalente a `git add`).
- **Realizar la Confirmación (Commit):** En el cuadro de texto superior que dice *Message*, escriba la descripción de sus cambios (ej. `feat: portafolio academico responsivo`) y haga clic en el botón de check **Commit** (o presione `Ctrl + Enter`).

2. Asistencia Inteligente con Google Antigravity

Google Antigravity puede ayudarle a redactar confirmaciones, configurar sus archivos de versionamiento y solucionar problemas de Git de manera ágil:

Generación de Mensajes de Commit Semánticos: En lugar de inventar mensajes ambiguos, pídale a Antigravity ayuda en el chat:

"Antigravity, genera un mensaje de commit semántico en formato Conventional Commits para mis cambios en index.html donde agregué la estructura y las secciones semánticas."

Creación de Reglas de Exclusión (.gitignore): Si está usando otros lenguajes o tecnologías en el futuro, puede solicitar configuraciones:

"Antigravity, genera un archivo .gitignore optimizado para un proyecto web con Node.js y VS Code."

Explicación de Comandos y Diagnósticos: Si se encuentra con un error en la terminal al realizar una fusión (merge) o conflicto:

"Antigravity, obtuve este error al ejecutar git push: [pegue el error aquí]. ¿Cómo puedo solucionarlo paso a paso?"

Paso 4: Sincronización y Publicación en GitHub Pages

1. Cree el repositorio en la nube:
2. Inicie sesión en [GitHub](#).
3. Haga clic en el botón **New** para crear un nuevo repositorio.
4. Asigne el nombre exacto de: portafolio-academico.
5. Configúrelo como **Public** (Público).
6. **Deje desmarcadas las opciones:** *Add a README file*, *Add .gitignore* y *Choose a license* (ya que creamos estos archivos de forma local).
7. Haga clic en **Create repository**.

Conecte su repositorio local con el remoto: GitHub le mostrará una serie de comandos. Copie y ejecute los siguientes en su terminal para renombrar la rama principal a `main`, asociar la URL remota y subir su código: `bash git branch -M main git remote add origin \ https://github.com/SU_USUARIO_GITHUB/portafolio-academico.git git push -u origin main` (Nota: Recuerde reemplazar `SU_USUARIO_GITHUB` por su nombre real de usuario de GitHub).

Active el Despliegue en GitHub Pages:

10. En la página de su repositorio en GitHub, ingrese a la pestaña **Settings** (Configuración) ubicada en el menú superior derecho.
11. En la barra lateral izquierda, busque la sección **Code and automation** y haga clic en **Pages**.
12. Bajo el menú **Build and deployment**, en la opción *Source*, asegúrese de que esté seleccionado **Deploy from a branch**.
13. En la opción *Branch*, cambie de *None* a **main** (y el directorio */root*), luego presione **Save** (Guardar).
14. Espere aproximadamente 1 a 2 minutos. Recargue la página de configuración y verá una caja de notificación verde en la parte superior que dice: > **Your site is live at** > https://su_usuario_github.github.io/portafolio-academico/
15. Ingrese al enlace generado y compruebe que su portafolio está en vivo para todo el mundo.

Parte 2: Depuración (Gotchas y Diagnósticos)

Gotcha 1: Error HTTP 404 al Acceder a la URL de GitHub Pages

Un error recurrente al desplegar en GitHub Pages es encontrarse con la pantalla de error "404 Site not found". Esto suele ser debido a dos particularidades técnicas:

1. **Sensibilidad a Mayúsculas/Minúsculas en Archivos de Entrada:** Los servidores web de producción que corren sobre Linux son sensibles a mayúsculas. Si usted nombró su archivo principal como `Index.html` (con "I" mayúscula) o `inicio.html`, el servidor web de GitHub Pages no lo reconocerá. GitHub Pages busca estrictamente un archivo de entrada llamado `index.html` completamente en minúsculas en el directorio raíz.
2. **Estructura de Carpetas Incorrecta:** El archivo `index.html` debe estar ubicado en la raíz del repositorio, no dentro de subcarpetas adicionales (como `src/index.html` o `html/index.html`), a menos que configure una ruta de publicación diferente.

Procedimiento de Diagnóstico y Corrección:

- [] Verifique el nombre del archivo en su editor de código. Si está nombrado de forma incorrecta (ej: `Index.html`), cámbielo a `index.html`.
- [] Confirme la modificación local ejecutando `git status`. Verá que Git marca el cambio como un renombrado.
- [] Agregue, confirme y empuje la corrección a GitHub: `bash git add . git commit -m "fix: index.html para hosting" git push origin main`
- [] Recargue el sitio web después de que la acción de GitHub Actions termine de ejecutarse para validar la carga exitosa.

Gotcha 2: Bloqueo de Autenticación al Hacer Push (Support for Password Authentication Removed)

Desde agosto del año 2021, GitHub eliminó el soporte de autenticación mediante la contraseña de su cuenta directamente por motivos de seguridad en la consola. Si intenta realizar un `git push` y escribe su contraseña usual, obtendrá el error: `fatal: Authentication failed for... O Support for password authentication was removed.`

Procedimiento de Diagnóstico y Corrección:

Para resolver esto, usted debe utilizar un **Token de Acceso Personal (PAT - Personal Access Token)** como contraseña en la consola:

- [] En su cuenta de GitHub, vaya a su foto de perfil en la esquina superior derecha y seleccione **Settings**.
- [] En el menú lateral izquierdo inferior, haga clic en **Developer Settings**.
- [] Seleccione **Personal access tokens** y luego **Tokens (classic)**.
- [] Haga clic en **Generate new token -> Generate new token (classic)**.
- [] Ingrese una descripción corta en *Note* (ej: "Token Consola Git"), marque el check de permisos para **repo** (esto da permisos para leer y escribir repositorios) y seleccione una fecha de expiración.
- [] Presione **Generate token** en el fondo de la pantalla.

- [] **IMPORTANTE:** Copie el token generado y guárdelo en un lugar seguro. No podrá volver a verlo una vez cierre la pestaña.
- [] Al ejecutar `git push` de nuevo en la terminal, cuando el sistema le solicite la contraseña (o cuadro de diálogo del Credential Manager), **pegue el token generado** en lugar de su contraseña tradicional.

Parte 3: Reto Autónomo (Sin Solución Explícita)

Para consolidar sus habilidades en control de versiones y estilos CSS responsivos, usted debe implementar una nueva funcionalidad trabajando bajo el flujo de trabajo estructurado de ramas (**Feature Branching**).

Requerimientos del Reto:

1. **Creación de la Rama de Trabajo:** Antes de editar su código, cree y cámbiese a una nueva rama llamada `feature-habilidades` en su terminal: `bash git checkout -b feature-habilidades`
2. **Modificación del HTML (`index.html`):**
3. Agregue una nueva sección con el identificador `habilidades` (`<section id="habilidades">`) justo antes de la etiqueta `main` de proyectos.
4. Diseñe una lista de habilidades que represente las tecnologías aprendidas en el curso (ej: HTML5, CSS3, Flexbox, Grid, Git, Java).
5. Estructure estas habilidades dentro de un contenedor utilizando `div`s individuales con la clase `skill-tag`.
6. **Modificación de los Estilos (`styles.css`):**
7. Estilice la sección y aplique **Flexbox** a las etiquetas (`skill-tag`).
8. Utilice la propiedad `flex-wrap: wrap` para que las etiquetas se acomoden en múltiples líneas si el espacio horizontal se reduce.
9. Añada colores de acento utilizando variables CSS, bordes redondeados y un efecto de `hover` sutil que cambie el color de fondo o el color de borde de cada etiqueta al pasar el mouse por encima.

10. Flujo de Confirmación e Integración de Git:

11. Una vez completados los cambios, guarde los archivos y añádalos al área de preparación.
12. Realice un commit en su rama de desarrollo: `bash git commit -am "feat: agregar seccion habilidades"`
13. Regrese a la rama principal (main): `bash git checkout main`
14. Realice la fusión (merge) de los cambios de su rama funcional a la rama principal: `bash git merge feature-habilidades`
15. Suba los cambios actualizados al servidor en GitHub: `bash git push origin main`
16. Borre la rama de desarrollo local una vez integrada con éxito: `bash git branch -d feature-habilidades`

Verificación del Éxito:

- Al ingresar a la URL pública de su portafolio en GitHub Pages, debe desplegarse la nueva sección de Habilidades de forma responsiva.
- En su terminal, ejecute `git log --oneline --graph`. Deberá observarse con claridad el historial secuencial de commits y la integración limpia de la rama `feature-habilidades` en la rama `main`.
- Reduzca el tamaño de la ventana de su navegador; valide que las etiquetas de habilidades se reordenen automáticamente a una fila inferior gracias al `flex-wrap` sin desbordar los límites laterales de la tarjeta o del contenedor.